

Discrete Mathematics

Jaysen Tsao

1	Speaking Mathematically	2
1.1	Introduction to Variables	2
1.2	Introduction to Sets	4
1.3	Introduction to Relations & Functions	7
1.4	Introduction to Graphs	8
2	Logic with Statements	9
2.1	Logical Form & Equivalence	9
2.2	Conditional Statements	11
2.3	Valid & Invalid Arguments	12
2.4	Digital Logic Circuits	14
2.5	Number Systems & Addition Circuits	14
3	Logic with Predicates & Quantifiers	15
3.1	Predicates & Quantifiers	15
4	Number Theory & Proofs	15
5	Sequences, Induction, & Recursion	15
6	Set Theory	15
7	Properties of Functions	15
8	Properties of Relations	15
9	Probability Theory	15
10	Graph Theory	16
10.1	Trails, Paths, & Circuits	16
10.2	Matrix Representations of Graphs	17
10.3	Graph Isomorphisms	18
10.4	Trees	19

1 Speaking Mathematically

1.1 Introduction to Variables

- A **variable** is a placeholder used to represent an unknown or arbitrary value.
- Variables can be introduced to reduce repetition or clarify relationships between quantities.

For example, instead of asking:

is there a number where doubling it and adding 3 gives us the same result as squaring it?

we can introduce a variable x and ask:

is there a number x such that $2x + 3 = x^2$?

- A variable in the context of computers represent a named storage location (e.g. a memory address) that holds a value.

Types of Mathematical Statements

- A **universal statement** makes a claim about all elements of particular nature. It is often expressed using phrases like “for all” or “for every.”

For example, “**for all** integers n , $n + 0 = n$ ” is a universal statement.

- A **conditional statement** asserts that if one statement, called the **hypothesis**, is true, then another statement, called the **conclusion**, is also true. It is often expressed in the form “if...then...”

For example, “**if** n is an even integer, **then** n^2 is also even” is a conditional statement.

- An **existential statement** claims that there is at least one thing that satisfies a certain property. It is often expressed using phrases like “there exists” or “there is at least one.”

For example, “**there exists** an integer n such that $n^2 = 4$ ” is an existential statement.

Universal Conditional Statements

- A **universal conditional statement** combines both a universal statement (which generalizes over many things) and a conditional statement (which specifies a condition for the generalization), applying the claim only to elements “in the set” that meet the condition.
 - This allows us to specify the condition under the assumption that the element has “satisfied” the universal statement check. For example, if we say “for all integers ...,” our check will only consider integers.
- These statements are often expressed in the form “for all...if...then...”

For example, “**for all** integers n , **if** n is even, **then** n^2 is also even” is a universal conditional statement.

Universal Existential Statements

- A **universal existential statement** combines both a universal statement (which generalizes over many things) and an existential statement (which asserts the existence of at least one thing), stating that out of everything which may satisfy the universal condition, there is at least one that meets the existential claim.
- These statements are often expressed in the form “for all...there exists...”

For example, “**for all** integers n , **there exists** an integer m such that $m = n + 1$ ” is a universal existential statement.

Existential Universal Statements

- An **existential universal statement** combines both an existential statement (which asserts the existence of at least one thing) and a universal statement (which generalizes over many things), stating that there is at least one thing such that for all elements of a certain nature, a claim holds true.
- These statements are often expressed in the form “there exists...for all...”

For example, “**there exists** an integer k such that **for all** integers n , $n + k = n$ ” is an existential universal statement.

Types of Mathematical Statements

- Universal Statement: *for all ...* ($\forall \dots$)
- Conditional Statement: *if ... then ...* ($\dots \Rightarrow \dots$)
- Existential Statement: *there exists ...* ($\exists \dots$)
- Universal Conditional Statement: *for all ... if ... then ...* ($\forall \dots, \dots \Rightarrow \dots$)
- Universal Existential Statement: *for all ... there exists ...* ($\forall \dots, \exists \dots$)
- Existential Universal Statement: *there exists ... such that for all ...* ($\exists \dots$ s.t. $\forall \dots$)

1.2 Introduction to Sets

- A **set** is a well-defined collection of *distinct* objects.
 - The objects in a set are called **elements** or **members** of the set.
- The notation $x \in S$ indicates that x is an element of the set S .
 - Similarly, $x \notin S$ indicates that x is *not* an element of the set S .
- **Axiom of Extension:** a set is defined solely by its elements, and the order or repetition of those elements does not matter.
 - For example, the sets $\{1, 2, 3\}$, $\{3, 2, 1\}$, and $\{1, 2, 2, 3\}$ are all considered the same set.
- The **cardinality** of a set is the number of elements in the set, denoted by $|A|$ for a set A .

Describing Sets

- **Set-roster notation** is a way to specify a set by explicitly listing its elements between curly braces.
 - For example, the set of primary colors can be represented as $C = \{\text{red, blue, yellow}\}$.
 - Ellipses (...) can be used to indicate:
 - a continuing pattern, such as in $\{1, 2, 3, \dots, 100\}$
 - an infinite set, such as in $\{1, 2, 3, \dots\}$ or $\{\dots, -2, -1, 0, 1, 2, \dots\}$
- **Set-builder notation** is a way to specify a set using a condition that its elements satisfy.
 - This notation takes the form: $A = \{x \in S \mid P(x)\}$ where:
 - x is a variable we introduce to represent an arbitrary element of S
 - S is a larger set from which we are selecting elements
 - $P(x)$ is the condition that elements must satisfy to be included in set A
 - In English, set A is *the set of elements x in S such that $P(x)$ holds true*.
 - For example, the set of real numbers between 1 and 3 (inclusive) can be represented as $A = \{x \in \mathbb{R} \mid 1 \leq x \leq 3\}$.
- Common numeric sets are given special symbols:
 - The set of natural numbers, $\mathbb{N} = \{1, 2, 3, \dots\}$ (may or may not include 0)
 - The set of integers, $\mathbb{Z} = \{\dots, -2, -1, 0, 1, 2, \dots\}$
 - The set of rational numbers, $\mathbb{Q}$
 - The set of real numbers, $\mathbb{R}$

We can also add superscripts to further narrow these sets, for example:

- $\mathbb{R}^+$ denotes the set of positive real numbers.
- $\mathbb{Z}^-$ denotes the set of negative integers.
- $\mathbb{R}^{\text{nonneg}}$ denotes the set of non-negative real numbers.

- The **empty set** is also given a special symbol: $\emptyset = \{\}$.

Relations Between Sets

- The relation between sets is described using *subsets* and *supersets*.
 - A set A is a **subset** of set B , denoted $A \subseteq B$, if for every element $x \in A$, $x \in B$.
 - A is a **proper subset** of B , denoted $A \subset B$, if $A \subseteq B$ and there exists a $y \in B$ such that $y \notin A$
 - A set B is a **superset** of set A , denoted $B \supseteq A$, if $A \subseteq B$.
 - B is a **proper superset** of A , denoted $B \supset A$, if $A \subset B$.

Subset

A set A is a **subset** of set B , denoted by $A \subseteq B$, if every element of A is also an element of B :

$$(\forall x \in A, x \in B) \iff A \subseteq B.$$

A is a **proper subset** of B , denoted by $A \subset B$, if $A \subseteq B$ and there exists at least one element in B that is not in A :

$$(A \subseteq B) \wedge (\exists y \in B \text{ s.t. } y \notin A) \iff A \subset B.$$

- We can also say $A \not\subseteq B$ to indicate that A is *not* a subset of B .
 - This means there exists at least one element in A that does not belong to B .

Cartesian Products

- The **Cartesian product** of two sets A and B , denoted by $A \times B$, is the set of all ordered pairs (a, b) where $a \in A$ and $b \in B$.
 - Formally, $A \times B = \{(a, b) \mid a \in A \wedge b \in B\}$.
 - For example, if $A = \{1, 2\}$ and $B = \{x, y\}$, then $A \times B = \{(1, x), (1, y), (2, x), (2, y)\}$.
 - The cardinality of the Cartesian product is $|A \times B| = |A| |B|$.
- An **ordered n -tuple** is a sequence of n elements treated as a distinct object, denoted by $(a_1, a_2, \dots, a_n)$, where the order and count of elements matters.
 - The Cartesian product can be extended to multiple sets, such as $A_1 \times A_2 \times \dots \times A_n$, resulting in ordered n -tuples.
 - Formally, $A_1 \times A_2 \times \dots \times A_n = \{(a_1, a_2, \dots, a_n) \mid a_i \in A_i \text{ for each } i = 1, 2, \dots, n\}$.
 - The cardinality of this Cartesian product is $|A_1 \times A_2 \times \dots \times A_n| = |A_1| |A_2| \dots |A_n|$.
 - Two n -tuples $(a_1, a_2, \dots, a_n)$ and $(b_1, b_2, \dots, b_n)$ are considered equal if and only if $a_i = b_i$ for all i from 1 to n .
- Cartesian products are neither commutative nor associative. That is, if A , B , and C are sets, then:
 - $A \times B \neq B \times A$ (since pairs are ordered)
 - $A \times B \times C \neq (A \times B) \times C \neq A \times (B \times C)$ (since the grouping affects the structure of the tuples)

Strings

- In mathematics, a **string** is a finite sequence of symbols from a given set called an **alphabet**.
 - The set of all strings that can be formed from an alphabet A is denoted by A^* .
- The **length** of a string is the number of symbols it contains.
 - The set of all strings of length n from an alphabet A is denoted by A^n .
 - The cardinality of the set of all strings of length n from an alphabet A is $|A^n| = |A|^n$.

Fixed-Length Strings

A **string** wrapped over A of length n is an ordered sequence of n elements from the **alphabet** A . The set of all strings of length n that can be formed from the alphabet A is denoted by:

$$A^n = \underbrace{A \times A \times \dots \times A}_{n \text{ times}}$$

- For example, $\mathbb{R}^2 = \mathbb{R} \times \mathbb{R}$ can be interpreted as the set of all strings of length 2 from the alphabet $\mathbb{R}$.
- **Null strings** or **empty strings** are strings of length 0, denoted by ε (in the textbook it's λ)
 - The set of all null strings over any alphabet A is $A^0 = \{\varepsilon\}$.
- **Bit strings** wrap over the alphabet $\{0, 1\}$.

1.3 Introduction to Relations & Functions

Set Relations

A **relation** R from set A to set B is a subset of the Cartesian product $A \times B$:

$$R \subseteq A \times B.$$

We call A the **domain** of the relation and B the **codomain** of the relation.

- If $(a, b) \in R$, we say that “ a is related to b by R ,” denoted by $a R b$.
- If $(a, b) \notin R$, we say that “ a is *not* related to b by R ,” denoted by $a \not R b$.

Arrow Diagrams

We can represent relations using **arrow diagrams**, where elements of the domain and codomain are represented as points, and an arrow drawn from a point A to a point B indicates that $(A, B) \in R$.

Functions

- A **function** f from A to B is a relation that associates each element of A with *exactly one* element of B . We can denote this relation as $f : A \rightarrow B$.
 - The set A is called the **domain** of the function.
 - The set B is called the **codomain** of the function.
 - The elements in B that are associated with elements in A is called the **range** of the function.
- If $(x, y) \in f$, then we can denote this relationship as $f(x) = y$.

Function

A **function** f from the **domain** A to the **codomain** B is a relation from A to B such that:

1. $\forall x \in A, \exists y \in B$ s.t. $(x, y) \in f$ (completeness)
2. $\forall x \in A, \forall y_1, y_2 \in B, \{(x, y_1), (x, y_2)\} \subseteq f \implies y_1 = y_2$ (uniqueness)

- Two functions f and g from A to B are equal, denoted by $f = g$, if and only if for every element $x \in A$, $f(x) = g(x)$.

Injections, Surjections, and Bijections

- A function is **one-to-one** or **injective** if different elements in the domain map to different elements in the codomain (no two inputs share the same output)
 - Formally, f is injective if $\forall x_1, x_2 \in A, f(x_1) = f(x_2) \implies x_1 = x_2$.
- A function is **onto** or **surjective** if every element in the codomain is the image of at least one element in the domain (all outputs are covered)
 - Formally, f is surjective if $\forall y \in B, \exists x \in A$ s.t. $f(x) = y$.
- A function is **bijective** if it is both one-to-one and onto.
 - Bijective functions have inverses that are also functions.

1.4 Introduction to Graphs

- A **graph** G is specified using two sets V and E where:
 - V is a non-empty set of **vertices**
 - E is a set of **edges**, where each edge is a pair of vertices from V
 - Edges can be unordered (as in a set $\{u, v\}$), as in an **undirected graph**
 - Edges can be ordered (as in a tuple (u, v)), as in a **directed graph** or **digraph**
- Each edge is associated with either one or two vertices $\in V$ called its **endpoints**.
 - If an edge e has endpoints $\{u, v\}$, then the **edge-endpoint function** maps e to $\{u, v\}$.
 - An edge with just one endpoint is called a **loop**.
 - Two edges with the same endpoints are called **parallel edges**.
 - Two vertices connected by an edge are called **adjacent vertices**.
- Two edges that share a common endpoint are called **adjacent edges**.
 - An edge is **incident** to its endpoints, and a vertex is **incident** to its edges.
- The **degree** of a vertex v , denoted by $\deg(v)$, is the number of edges incident to v , with loops counted twice.
 - A vertex with no incident edges (i.e., $\deg(v) = 0$) is called an **isolated vertex**.

2 Logic with Statements

2.1 Logical Form & Equivalence

- An **argument** is a sequence of statements where the last statement is called the **conclusion** and the preceding statements are called **premises**.
 - An argument is **valid** if the conclusion logically follows from the premises, meaning that if the premises are true, then the conclusion must also be true.
 - An argument is **invalid** if the conclusion does not logically follow from the premises.
- A **statement** or **proposition** is a declarative sentence that is either true or false, but not both.
- A **tautology** (**t**) is a statement that is *always* true. It is logically equivalent to the statement “true.”
 - For example, the statement “ P or not P ” is a tautology.
- A **contradiction** (**c**) is a statement that is *always* false. It is logically equivalent to the statement “false.”
 - For example, the statement “ P and not P ” is a contradiction.
- Two statements are **logically equivalent** if they have the same truth value in every possible scenario.
 - We denote logical equivalence between a statement P and another statement Q by writing $P \equiv Q$.

Compound Statements

- A **compound statement** is a statement formed by combining one or more statements using logical connectives such as “and,” “or,” and “not.”
- The following are common logical connectives (in the order they should be applied):
 - **Negation** ($\neg P$): the opposite truth value of statement P .
 - **Conjunction** ($P \wedge Q$): true if both P and Q are true; false otherwise.
 - **Disjunction** ($P \vee Q$): true if at least one of P or Q is true; false only if both are false.

Logical Equivalences

Name	Equivalence
Commutative Law	$P \vee Q \equiv Q \vee P$ $P \wedge Q \equiv Q \wedge P$
Associative Law	$(P \vee Q) \vee R \equiv P \vee (Q \vee R)$ $(P \wedge Q) \wedge R \equiv P \wedge (Q \wedge R)$
Distributive Law	$P \vee (Q \wedge R) \equiv (P \vee Q) \wedge (P \vee R)$ $P \wedge (Q \vee R) \equiv (P \wedge Q) \vee (P \wedge R)$
Identity Law	$P \vee \mathbf{c} \equiv P$ $P \wedge \mathbf{t} \equiv P$
Negation Law	$P \wedge \neg P \equiv \mathbf{c}$ $P \vee \neg P \equiv \mathbf{t}$
Double Negative Law	$\neg(\neg P) \equiv P$
Idempotent Law	$P \vee P \equiv P$ $P \wedge P \equiv P$
Absorption Law	$P \vee (P \wedge Q) \equiv P$ $P \wedge (P \vee Q) \equiv P$
De Morgan's Laws	$\neg(P \wedge Q) \equiv (\neg P) \vee (\neg Q)$ $\neg(P \vee Q) \equiv (\neg P) \wedge (\neg Q)$
Standard Negations	$\neg \mathbf{t} \equiv \mathbf{c}$ $\neg \mathbf{c} \equiv \mathbf{t}$

2.2 Conditional Statements

- A **conditional statement** is a logical statement that has the form “if P , then Q ,” where P is the **hypothesis** and Q is the **conclusion**. It is denoted by $P \Rightarrow Q$.
 - P is a **sufficient condition** for Q if $P \Rightarrow Q$ is true.
 - P is a **necessary condition** for Q if $\neg P \Rightarrow \neg Q$ (i.e. $Q \Rightarrow P$) is true.
- A conditional statement is always true if the hypothesis is false, regardless of the truth value of the conclusion. This is known as **true by default** or **vacuously true**.
- The conditional statement $P \Rightarrow Q$ is logically equivalent to $\neg P \vee Q$.
- The statement $P \vee Q \Rightarrow R$ is logically equivalent to $(P \Rightarrow R) \wedge (Q \Rightarrow R)$.

Biconditionals

- The statement “**only if** P , then Q ” is logically equivalent to “ Q only if P ” and is denoted by $Q \Rightarrow P$. It establishes a necessary condition for Q .
- A **biconditional statement** is a logical statement that has the form “ P if and only if Q ,” denoted by $P \Leftrightarrow Q$. It is true only when both P and Q have the same truth value (both true or both false).
 - P is a **necessary and sufficient condition** for Q if $P \Leftrightarrow Q$ is true.
 - $P \Leftrightarrow Q$ is logically equivalent to $(P \Rightarrow Q) \wedge (Q \Rightarrow P)$.

Related Statements to Conditional Statements

Let P and Q be statements. The following related statements can be derived from the conditional statement $P \Rightarrow Q$:

Name	Form	Logical Equivalence
contrapositive	$\neg Q \Rightarrow \neg P$	logically equivalent to $P \Rightarrow Q$
converse	$Q \Rightarrow P$	NOT logically equivalent
inverse	$\neg P \Rightarrow \neg Q$	NOT logically equivalent
negation	$P \wedge \neg Q$	logically equivalent to $\neg(P \Rightarrow Q)$

2.3 Valid & Invalid Arguments

- An **argument** is a sequence of statements where the last statement is called the **conclusion** and the preceding statements are called **premises**.
- An argument is **valid** if the conclusion logically follows from the premises, meaning that if the premises are true, then the conclusion must also be true.
- In a truth table, an argument is valid if there are no rows where all the premises are true and the conclusion is false.
 - Rows where all premises are true are called **critical rows**.

Example: Using a Truth Table to Determine Validity

Consider the argument:

$$\begin{aligned} p &\rightarrow q \vee \neg r \\ q &\rightarrow p \wedge r \\ \therefore p &\rightarrow r \end{aligned}$$

We can construct a truth table to evaluate the validity of this argument. The critical rows are those where both premises are true. If in all critical rows the conclusion is also true, then the argument is valid.

$$p \mid q \mid r \mid p \rightarrow q \vee \neg r \mid q \rightarrow p \wedge r \mid p \rightarrow r$$

- An argument with two premises and a conclusion is called a **syllogism**, where the first premise is called the **major premise** and the second premise is called the **minor premise**. There are two common syllogisms:
 - **Modus Ponens**: If $P \Rightarrow Q$ is true and P is true, then Q must be true.
 - **Modus Tollens**: If $P \Rightarrow Q$ is true and $\neg Q$ is true, then $\neg P$ must be true.

Summary of Inferences

Inference	Form	Inference	Form
Modus Ponens	$P \Rightarrow Q$ P $\therefore Q$	Elimination	$P \vee Q$ $\neg P$ $\therefore Q$
Modus Tollens	$P \Rightarrow Q$ $\neg Q$ $\therefore \neg P$	Transitivity	$P \Rightarrow Q$ $Q \Rightarrow R$ $\therefore P \Rightarrow R$
Generalization	$P \therefore P \vee Q$	Proof by Cases	$P \vee Q$ $P \Rightarrow R$ $Q \Rightarrow R$ $\therefore R$
Specialization	$P \wedge Q \therefore P$	Contradiction	$\neg P \Rightarrow c$ $\therefore P$
Conjunction	P Q $\therefore P \wedge Q$		

Fallacies

- A **fallacy** is an error in reasoning that leads to an invalid argument. Some common fallacies include:
 - Using ambiguous premises
 - Circular reasoning, assuming what is to be proved
 - Jumping to a conclusion
- The **converse error** is the fallacy of assuming that if $P \Rightarrow Q$ is true, then $Q \Rightarrow P$.
 - ex: *if it snows, there is no school. There is no school. Therefore, it is snowing*
- The **inverse error** is the fallacy of assuming that if $P \Rightarrow Q$ is true, then $\neg P \Rightarrow \neg Q$.
 - ex: *if it snows, there is no school. It is not snowing. Therefore, there is school.*
- An argument is **sound** if it is valid and all of its premises are *actually true*. Otherwise, it is **unsound**.

Contradictions

- If an argument leads to a contradiction, it means that the premises cannot all be true at the same time. This is often used in **proof by contradiction**.
 - That is, if $p \rightarrow c$, then it is valid to say $\neg p$.

2.4 Digital Logic Circuits

- A **black box** is a component in a digital circuit that has a specified behavior (input-output relationship) but whose internal workings are not visible.
 - The behavior of a black box can be described using an **input/output table**, which is essentially truth table.
- A variable that takes only one of two values is known as a **Boolean variable**. An expression composed of Boolean variables is called a **Boolean expression**.
- A **recognizer** is a circuit that takes an input and outputs 1 if the input satisfies a specific combination of inputs, and 0 otherwise. In other words, there is only one 1 in the output column of the input/output table.
- Two logic circuits are **equivalent** if they have equivalent input/output tables.

NAND and NOR gates

There are two special shorthand gates:

- The **NAND gate** acts like an AND gate followed by a NOT gate. It outputs 0 only when all inputs are 1, and outputs 1 otherwise.
 - The symbol for NAND is an up arrow ($p \uparrow q$), combining the bar for not and the wedge for and.
 - In the textbook, a bar is used ($p \mid q$), called the **Sheffer stroke**.
- The **NOR gate** acts like an OR gate followed by a NOT gate. It outputs 1 only when all inputs are 0, and outputs 0 otherwise.
 - The symbol for NOR is a down arrow ($p \downarrow q$), combining the bar for not and the vee for or. This is called a **Peirce arrow**.

Here are a few equivalencies:

- $\neg(p \wedge q) \equiv p \uparrow q \equiv \neg p \vee \neg q$
- $\neg(p \vee q) \equiv p \downarrow q \equiv \neg p \wedge \neg q$
- $\neg p \equiv p \uparrow p$

2.5 Number Systems & Addition Circuits

- A way to represent a signed integer in binary is using **two's complement**
 - To find the two's complement of a binary number, we can invert all the bits (complement) and add 1 to the result.
 - We can also interpret it as saying the last digit has a negative weight. Instead of representing 2^n , it represents -2^n .

3 Logic with Predicates & Quantifiers

3.1 Predicates & Quantifiers

Negations of Quantified Statements

$$\neg(\forall x \in S, P(x)) \equiv \exists x \in S \text{ s.t. } \neg P(x)$$
$$\neg(\exists x \in S \text{ s.t. } P(x)) \equiv \forall x \in S, \neg P(x)$$

4 Number Theory & Proofs

5 Sequences, Induction, & Recursion

6 Set Theory

7 Properties of Functions

8 Properties of Relations

9 Probability Theory

10 Graph Theory

10.1 Trails, Paths, & Circuits

- A **walk** in a graph is a sequence of vertices and edges where each edge connects the vertices immediately before and after it in the sequence.
 - walk $\rightarrow$ **trail** (no repeated edges) $\rightarrow$ **path** (no repeated edges & vertices)
 - **closed walk** $\rightarrow$ **circuit** (closed trail) $\rightarrow$ **simple circuit** (closed path)
 - for circuits/simple circuits, must contain at least one edge
 - for simple circuits, only the start/end vertex is repeated
 - a graph with m edges and n vertices has at least one circuit iff $m \geq n$
 - The **trivial walk** from a vertex v to itself is the walk that consists of just the vertex v and no edges. It is considered a closed walk but not a circuit or simple circuit.
- A walk can be denoted by listing the vertices and edges in order, for example: $v_1 e_1 v_2 e_2 v_3$.
 - The notation $v_1 v_2 v_3 \dots$ can be ambiguous if there are multiple edges between adjacent vertices.
 - The notation $e_1 e_2 e_3 \dots$ can be ambiguous for undirected graphs.
- A graph is **simple** if it is undirected and has no loops or parallel edges.

Connectedness

- A graph $H = (V_H, E_H)$ is a **subgraph** of a graph $G = (V_G, E_G)$ if $V_H \subseteq V_G$ and $E_H \subseteq E_G$.
- A graph is **connected** if there is a walk between every pair of vertices in the graph.
 - Formally, G is connected $\iff \forall$ vertices $u, v \in G, \exists$ walk from u to v
- A **connected component** of a graph is a maximal connected subgraph.
 - In other words, it is a connected subgraph that cannot be extended by including any additional vertices or edges from the original graph without losing its connectedness.
 - Formally, a subgraph C of a graph G is a connected component if and only if:
 - C is connected.
 - There is no connected subgraph C' of G such that $C \subset C'$.

Euler and Hamiltonian Circuits

- An **Euler trail** is a trail that passes through every edge and vertex of a graph at least once.
- An **Euler circuit** is an Euler trail that starts and ends at the same vertex.
 - A graph has an Euler circuit iff it is connected and every vertex has an even degree.
 - A graph has an Euler trail (but not an Euler circuit) iff it is connected and exactly two vertices u, v have odd degree and all other vertices have even degree. Then, the trail starts at u and ends at v .
- A **Hamiltonian path** is a path that visits each vertex exactly once.
 - A **Hamiltonian circuit** is a Hamiltonian path that is also a simple circuit.
- The **Traveling Salesman Problem** is the problem of finding a Hamiltonian circuit with the minimum total weight in a weighted graph.
 - *Which route should a salesman take to visit each city exactly once and return to the starting city, while minimizing the total distance traveled?*

10.2 Matrix Representations of Graphs

- An **adjacency matrix** A for a graph with n vertices is an $n \times n$ matrix where the entry a_{ij} indicates the number of edges between vertex i and vertex j .
 - For undirected graphs, the adjacency matrix is symmetric.
 - For directed graphs, the entry a_{ij} indicates the number of edges from vertex i to vertex j .
- The number of walks of length n from v_i to v_j in a graph is the ij^{th} entry of the matrix A^n , where A is the adjacency matrix of the graph.

10.3 Graph Isomorphisms

- Two graphs $G_1 = (V_1, E_1)$ and $G_2 = (V_2, E_2)$ are **isomorphic** if there exists a bijection $f : V_1 \rightarrow V_2$ such that any two vertices u and v in V_1 are adjacent in G_1 if and only if $f(u)$ and $f(v)$ are adjacent in G_2 .
 - In other words, the structure of the graphs is the same, even if the labels of the vertices are different.
 - If we can map each vertex of G_1 to a unique vertex of G_2 such that the adjacency relationships are preserved, then the two graphs are isomorphic.
- Graph isomorphisms follow an equivalence relation:
 - Reflexive: Every graph is isomorphic to itself.
 - Symmetric: If G_1 is isomorphic to G_2 , then G_2 is isomorphic to G_1 .
 - Transitive: If G_1 is isomorphic to G_2 and G_2 is isomorphic to G_3 , then G_1 is isomorphic to G_3 .
- A graph **invariant** is a property that remains unchanged under graph isomorphisms. The following are the invariants listed in the textbook:
 1. number of vertices and edges
 2. existence of n vertices of a certain degree
 3. existence of a circuit of a certain length
 4. existence of n simple circuits of a certain length
 5. connectedness
 6. existence of Euler circuits
 7. existence of Hamiltonian circuits
- We can prove that two graphs are not isomorphic by showing that they have different invariants. For example, a graph with 5 vertices *cannot* be isomorphic to a graph with 4 vertices.

However, we cannot prove that two graphs are isomorphic just by showing that they have the same invariants, since there may be non-isomorphic graphs that share the same invariants.

10.4 Trees

- A **tree** is a graph that is both circuit-free and connected.
 - A tree with n vertices has exactly $n - 1$ edges.
- The **total degree** of a graph is the sum of the degrees of all its vertices.
 - Each edge in a tree connects exactly two vertices, i.e. a tree with E edges has a total degree of $2E$.
 - Since a tree with n vertices has $n - 1$ edges, the total degree of a tree is $2(n - 1)$.
 - The fact that the total degree is an even number is called the **Handshaking Lemma**.
- A vertex in a tree with degree 1 is called a **leaf** or **terminal vertex**.
 - All nontrivial trees have at one leaf.
 - If a vertex in a tree is not a leaf, it is called a **branch vertex** or **internal vertex**.
- A graph that is circuit-free and *not* connected is called a **forest**.
- A **trivial tree** is a tree with a single vertex and no edges.

Rooted Trees

- A **rooted tree** is a tree in which one vertex has been designated as the **root**.
 - The **level** of a vertex in a rooted tree is the length of the unique path from the root to that vertex.
 - The **height** of a rooted tree is the maximum level of any vertex in the tree.
- For any vertex in a rooted tree, the vertices adjacent to it that are one level farther away from the root are called its **children**, and the vertex one level closer to the root is called its **parent**.
 - A vertex with no children is a **leaf** or **terminal vertex**.
 - A vertex with at least one child is a **branch vertex**.
 - In other words, if u is a child of v , then v is the parent of u .
 - Two vertices with the same parent are called **siblings**.
- If the path from the root to a vertex v contains the vertex u , then u is an **ancestor** of v , and v is a **descendant** of u .
 - The **subtree** rooted at a vertex v is the tree consisting of v and all its descendants.

Binary Trees

- A **binary tree** is a rooted tree in which each vertex has at most two children.
 - The children of a vertex in a binary tree are referred to as the **left child** and the **right child**.
 - The **left subtree** of a vertex is the subtree rooted at its left child, and the **right subtree** is the subtree rooted at its right child.
- A **full binary tree** is a binary tree in which every vertex has either 0 or 2 children.
 - A full binary tree with n leaves has exactly $2n - 1$ vertices.

Spanning Trees

- A **spanning tree** of a graph G is a subgraph of G that is a tree and contains all the vertices of G .
 - A graph may have multiple spanning trees, but every spanning tree of a graph has the same number of edges, which is equal to the number of vertices in the graph minus one.
- A **minimum spanning tree** of a weighted graph is a spanning tree with the minimum total weight among all spanning trees of the graph.